

Introduction to Unix systems

Lecture 02

Computer Physics

15 October 2026

Files, permissions and the system environment

Jacek Gatlik

email: jacek.gatlik@agh.edu.pl
www: https://galaxy.agh.edu.pl/~jgatlik/

Faculty of Physics and Applied Computer Science,
AGH University of Krakow

Outline

Filesystem hierarchy

Text editors in the terminal

Users and groups

Ownership and permissions

Links and metadata

Environment variables

Shell configuration

Section 1

Filesystem hierarchy

One directory tree, rooted at /

```
/
|-- home/
|-- etc/
|-- usr/
|-- var/
|-- tmp/
|-- dev/
|-- proc/
`-- sys/
```

Selected top-level directories.

- The visible hierarchy begins at **/**, the root directory.
- Other filesystems are attached at directories called **mount points**.
- A path may lead to a local disk, network storage or a virtual filesystem provided by the kernel.
- Standard directory names organise files by purpose. Cluster storage may use additional, site-specific paths.

One tree does not mean one disk

On a cluster, your home directory and simulation output may reside on different storage systems, with different performance, quotas and retention rules.

System software, configuration and changing data

Directory	Main purpose
`/usr`	Installed software: commands in `/usr/bin`, libraries in `/usr/lib`, shared data in `/usr/share`.
`/usr/local`	Software installed locally by the administrator.
`/etc`	System-wide configuration, e.g. `/etc/passwd`.
`/var`	Changing system data: logs, caches and service state.
`/opt`	Additional application packages.
`/boot`	Files used during the boot process.

On many modern distributions, `/bin`, `/sbin` and `/lib` are symbolic links into `/usr`:

```
1 ls -ld /bin /sbin /lib
```

Remember: `/usr` is a software hierarchy; your personal files belong in your home or project storage.

Home directories and temporary storage

- **Your home directory:** often `/home/username`. Use `~` or `$HOME` instead of assuming a particular path.
- **/root:** normally the root user's home directory; `/` is the root of the directory tree.
- **/tmp:** temporary files that may be removed automatically, including during reboot. It may use RAM-backed storage.
- **/mnt and /media:** conventional locations for temporary mounts and removable media; the actual layout varies.

```
cd "$HOME"
pwd
mkdir -p unix-course/lecture02
```

On a computing cluster

A node's `/tmp` need not be visible from other nodes. Use the storage documented for your cluster and workload.

Special filesystems: interfaces to the running system

Directory	What it exposes or stores
`/dev`	Device interfaces and special files, such as terminals and `/dev/null`.
`/proc`	Kernel and process information, e.g. memory statistics and per-process directories.
`/sys`	Devices, drivers and other kernel objects, including information and control attributes.
`/run`	Volatile runtime state maintained by services; normally recreated after boot.

Read-only exploration:

```
ls -l /dev/null
head -n 5 /proc/meminfo
ls /sys/class/net
```

For simulations: `/proc/meminfo` describes system memory, not your job's allocated memory limit. These paths do not necessarily represent stored disk data.

Organising a simulation project

```
wave-simulation/
|-- README.md
|-- src/
|-- input/
|-- runs/
|   |-- run-001/
|   `-- run-002/
|-- analysis/
`-- figures/
```

An example organisation, not a Unix standard.

- Keep source code and original inputs separate from generated data.
- Give each run its own output directory. Record parameters, random seed and code version.
- On HPC systems, large run directories belong on the designated work filesystem.

Estimate output before starting a simulation

One uncompressed scalar field on a 256^3 grid, using 8-byte values: $256^3 \times 8$ bytes $= 128$ MiB.

Saving 1000 snapshots requires **125 GiB** before metadata, extra fields and checkpoints.

Checking filesystem capacity and directory size

Run these commands from a small project directory:

```
df -h .
df -i .
du -sh .
du -h --max-depth=1 .
```

- `df -h`: space used and available on the filesystem containing the current directory.
- `df -i`: inode usage, where supported. Files and directories consume these filesystem objects.
- `du -sh`: total disk usage of the directory tree. `--max-depth=1` also reports immediate subdirectories.
- Recursive `du` scans can be expensive on shared storage; inspect a relevant project, not all of scratch.

Free filesystem space is not your quota

A shared filesystem may have terabytes free while your account or project has already reached its own limit.

Disk quotas: limits on space and file count

A **quota** limits storage use by a user, group or project on a particular filesystem.

What can be limited?

- **Space:** total allocated storage.
- **Inodes / file count:** the number of filesystem objects, including directories.

Many tiny output files can exhaust the file-count quota while using little space.

How are limits enforced?

- **Soft limit:** may be exceeded temporarily during a configured grace period.
- **Hard limit:** further allocation is blocked.

After the grace period expires, the soft limit also blocks further allocation.

```
1 quota -s
```

Where supported and configured; clusters may require their own tools, such as `hpc-fs`.

Simulation failure: `Disk quota exceeded` can prevent saving a checkpoint even when `df` shows free space.

Ares: choosing storage for your data

Use the paths supplied through environment variables:

Location	Intended use
`$HOME`	Personal files, configuration, source code and scripts; not large simulation output.
`$SCRATCH`	Temporary computation data: run directories, intermediate files and large output awaiting transfer.
`$PLG_GROUPS_STORAGE/<groupname>`	Shared project data, available for the duration of the grant; subject to the assigned allocation.

Check your actual paths and limits on Ares:

```
echo "$HOME"
echo "$SCRATCH"
echo "$PLG_GROUPS_STORAGE"
hpc-fs
```

`<groupname>` means your assigned group. Storage paths and quotas can differ between PLGrid systems.

Ares and PLGrid: storage rules for simulation work

Ares scratch cleanup

The documented policy automatically removes scratch data older than **30 days** and job directories older than **7 days**. Transfer valuable results promptly; scratch is temporary storage.

- **Before a run:** use `hpc-fs` to check space and file-count limits. Leave room for checkpoints and simultaneous jobs.
- **During a run:** use `$SCRATCH` or the documented `$TMPDIR` for temporary data, not `/tmp`. Avoid many tiny files; consider grouped arrays within a file, e.g. HDF5.
- **After a run:** verify and copy needed results to project storage; maintain an independent backup. Remove intermediate data once it is no longer needed.
- **Across PLGrid:** check each centre's storage and retention policy. Project storage tied to a grant is not a permanent archive.

Cyfronet: Ares documentation Storage environment variables

Section 2

Text editors in the terminal

nano: edit a file in the terminal

Edit simulation inputs, scripts and job files, including over SSH:

```
nano simulation.ini
```

Type text directly; move the cursor with the arrow keys.

Shortcut	Action with default bindings
`Ctrl+O`, then `Enter`	Save; confirm the filename.
`Ctrl+X`	Exit; answer the prompt if changes are unsaved.
`Ctrl+W`	Search: type the text, then press `Enter`.
`Alt+U` / `Alt+E`	Undo / redo.
`Ctrl+G`	Open help.

Read the shortcuts at the bottom of the screen

`^O` means `Ctrl+O`; `M-U` means `Alt+U`. Local configuration can change the bindings.

Vim: know which mode you are in

```
vim simulation.ini
```

For a first editing session:

- Vim starts in **Normal mode**: keys are commands.
- Press `i` for **Insert mode**: type your text.
- Press `Esc` to return to Normal mode.
- From Normal mode, `:` opens the command line.

Finish each command with `Enter`:

Command	Action
`:w`	Save.
`:q`	Quit; refuses if changes are unsaved.
`:wq`	Save and quit.
`:q!`	Discard unsaved edits; quit.

When typing behaves unexpectedly

Press `Esc` before issuing a Normal-mode command. `:q!` does not undo changes already saved with `:w`.

Vim: move, search and correct mistakes

The following commands work in **Normal mode**:

Keys	Action
`h j k l`	Move left, down, up, right.
`w` / `b`	Move forward / backward by word starts.
`gg` / `G`	Go to the first / last line.
`/steps`, then `Enter`	Search forward for `steps`.
`n` / `N`	Repeat the search in the same / opposite direction.
`dd`	Delete the current line.
`u` / `Ctrl+R`	Undo / redo.

Case and sequence matter

`n` and `N` are different commands. `dd` means press `d` twice, in sequence. Return to Insert mode with `i` when you want to type text.

Further practice: run `vimtutor` from the shell, if installed.

Live example: edit a simulation input

Example contents of `simulation.ini`:

```
steps = 1000
dt = 0.01
output_every = 100
seed = 42
```

1. Open the file and find `output_every`.
2. Change its value from `100` to `250`.
3. Make a deliberate editing mistake, then undo it.
4. Save, exit and reopen the file to verify the change.

Repeat the same task using the other editor.

Section 3

Users and groups

Users: names, identifiers and running processes

- A **user account** has a name and a numeric **user ID (UID)**. Files store numeric ownership IDs; commands usually display the corresponding names.
- Personal accounts identify people. Service accounts let system services run under separate identities.
- A process runs with user and group credentials. Its **effective identity** normally determines its access to files.
- **root** has UID 0 and administrative privileges. `sudo` can run an authorised command as another user, usually root; access depends on local policy.

```
whoami
id
```

`whoami` prints your effective username; `id` displays user and group IDs.

On a research system

Use your own account for simulations and groups for shared project access.

Groups and shared research projects

Each group has a name and a numeric **group ID (GID)**. An illustrative `id` result:

```
uid=1001(student) gid=1001(students) groups=1001(students),2000(physics)
```

- **Primary group:** here, `students`. Newly created files normally use the process's effective group; a setgid directory can change this rule.
- **Supplementary groups:** here, `physics`. These also participate in group-based permission checks.
- Each file has **one owning user and one owning group**. A user can belong to several groups.
- Group changes may require a new login session before your shell and new jobs see the updated membership.

```
1 id -gn
2 id -nG
3 getent group physics
```

First: effective group name; second: all current group names; third: the named group in the system's account database. Example account and group names will differ on your system.

Section 4

Ownership and permissions

Reading ownership and permissions

An illustrative `ls -l results.dat` entry:

```
-rw-r----- 1 student physics 8192 Oct 12 10:00 results.dat
```

Type	Owner permissions	Group permissions	Other permissions
-	rw-	r--	---

- Type: `-` regular file, `d` directory, `l` symbolic link. Owner: `student`; group: `physics`.
- Use `ls -ld run-001` to inspect a directory itself, and `ls -ln` to display numeric ownership IDs.

```
chgrp physics results.dat
sudo chown alice:physics results.dat
```

Who may change ownership? An ordinary owner can change the group to one they belong to. Changing the owning user requires administrative privilege; the second command is an administrator's example.

Read, write and execute: files versus directories

Bit	Regular file	Directory
`r`	Read its contents.	List the names of its entries.
`w`	Modify or truncate its contents.	Create, remove or rename entries; normally also requires `x`.
`x`	Execute it as a program, if its format and the system allow it.	Traverse the directory and look up entries by name.

A directory is a collection of names

Its permissions control access to those names. They do not automatically grant permission to read or modify the contents of the files inside.

Shell scripts: `./run.sh` normally needs read and execute permission. `bash run.sh` needs read permission, but not the script's execute bit.

Directory permissions along a path

To read `/project/run-001/results.dat`, you need:

- Search permission, `x`, on every directory in the path: `/`, `/project` and `/project/run-001`.
- Read permission, `r`, on `results.dat` itself.

Directory with x but no r

You may access a known filename, if the file's permissions allow it, but cannot list the directory's names.

Directory with r but no x

You may list names, but cannot normally open those entries or obtain their metadata through this directory.

Creating, deleting and renaming: normally require `w+x` on the containing directory. Renaming between directories requires this on both.

A read-only file can still be deleted if the parent directory permits it. The sticky bit adds a restriction discussed later.

Which permission class applies?

For the ordinary permission-bit model, select **one class**:

1. Does the process's effective UID match the file owner? Use the **owner** bits.
2. Otherwise, does the file's group match the effective group or any supplementary group? Use the **group** bits.
3. Otherwise, use the **other** bits.

`-r--rw----  alice  physics  results.dat`

User	Selected class	Access to contents
Alice, also in `physics`	Owner: `r--`	Read only
Bob, in `physics`	Group: `rw-`	Read and write
Carol, outside `physics`	Other: `---`	Neither

Permissions are not added across classes. Alice does not fall back to group permissions when owner write is absent.

Examples assume no extended ACLs or administrative privilege, and sufficient directory traversal permissions.

Changing permissions: symbolic and octal notation

The owner or a privileged process can use `chmod`. Having write permission alone does not authorise it.

Symbolic notation

- Who: `u` owner, `g` group, `o` others, `a` all three.
- Operation: `+` add, `-` remove, `=` set exactly.

```
chmod u+x run.sh
chmod g+w results.dat
chmod o= results.dat
chmod u=rw,g=r,o= results.dat
```

Octal notation: one digit per class

$r = 4, \quad w = 2, \quad x = 1$

`---`	0	`r--`	4
`--x`	1	`r-x`	5
`-w-`	2	`rw-`	6
`-wx`	3	`rwx`	7

```
chmod 640 results.dat
chmod 750 run.sh
```

Calculate each triplet separately

For `rwxr-x---`: owner $4 + 2 + 1 = 7$, group $4 + 1 = 5$, others 0. The mode is **750**: owner, group and others, in that order.

umask: removing permissions at creation time

A program requests an initial mode. Common requests are:

- **Regular file:** `666` = `rw-rw-rw-`.
- **Directory:** `777` = `rwxrwxrwx`.

A mask bit of 1 removes the corresponding permission

`actual = requested & (~umask)`

Bitwise AND with the inverted mask, restricted to the nine `rwx` bits. This is **not ordinary subtraction**.

```
1 umask
2 umask 027
```

- No argument: display the mask. With an argument: set it; future child processes inherit it.
- Existing files are unchanged; explicit `chmod 640 file` is not filtered through the mask.
- A mask cannot add permission. Even `umask 000` does not add `x` to a file requested as `666`.

Programs may request stricter modes. Examples assume no default ACL on the parent directory.

Calculating umask 027 bit by bit

Write each octal digit as three binary bits in `rwx` order:

	Owner	Group	Others
Mask `027`	`000`	`010`	`111`
Inverted mask	`111`	`101`	`000`
File request `666`	`110`	`110`	`110`
File AND inverted mask	`110` = 6	`100` = 4	`000` = 0
Directory request `777`	`111`	`111`	`111`
Directory AND inverted mask	`111` = 7	`101` = 5	`000` = 0

Interpret the mask before calculating

`0`: keep the owner's requested permissions; `2`: remove group write; `7`: remove all permissions for others.

Result: file `640` = `rw-r-----`; directory `750` = `rwxr-x---`.

Choosing a mask and checking the result

Typical creation requests

Mask	File: 666	Dir: 777
022	644	755
027	640	750
077	600	700
002	664	775
007	660	770
003	664	774

Try it with fresh names

```
(
  umask 027
  mkdir mask-demo
  touch mask-demo/results.dat
  ls -ld mask-demo
  ls -l mask-demo/results.dat
)
```

Parentheses create a subshell, so the mask of your original shell stays unchanged. Use a directory name that does not already exist.

Why subtraction fails: mask 003, others' file bits

Requested `6 = 110`; mask `3 = 011`; inverted mask `100`.
`110 AND 100 = 100 = 4`: the result is **664**, not 663.

`touch` does not reset permissions of an existing file.

Special permission bits: the fourth octal digit

A four-digit mode has the form **special–owner–group–others**. The first digit adds independent flags: $4 + 2 + 1$.

Value	Flag	Example	Displayed mode
4	setuid	`4755`, executable	`-rwsr-xr-x`
2	setgid	`2770`, directory	`drwxrws---`
1	sticky	`1777`, directory	`drwxrwxrwt`

The special flags share display positions with execute bits:

- `s`/`S`: setuid in the owner's `x` position; setgid in the group's `x` position.
- `t`/`T`: sticky in the others' `x` position.
- Lowercase `s`, `t`: the corresponding `x` is also set. Uppercase `S`, `T`: it is absent.

Decode both the flag and the ordinary permissions

`2740` gives `drwxr-S---` for a directory: setgid is set, but group search permission is missing.

setuid and setgid on executable programs

Normally, a program runs with the effective identity of its caller. For an executable binary, special bits can change this:

Flag	Effect when the program starts
setuid	Effective UID becomes the UID of the executable's owner.
setgid	Effective GID becomes the executable's owning GID.

```
1 ls -l /usr/bin/passwd
```

- `passwd` is commonly a root-owned setuid program: it performs controlled updates to protected password data.
- **setuid does not always mean root.** The executable's owner determines the effective UID.
- Linux ignores setuid/setgid bits on interpreter scripts. Mount options and security settings can also suppress their effect.

For scientific work: simulation programs normally need ordinary execute permission. Special execution privileges require deliberate system administration.

setgid directories: sharing simulation output

Assume you belong to `physics` and create a new project directory:

```
mkdir shared-runs
chgrp physics shared-runs
chmod 2770 shared-runs
```

Directory mode: `drwxrws---`.

- New files inherit the directory's **group**. New subdirectories also inherit its **setgid bit**.
- Ordinary `rwx` permissions still depend on the requested creation mode and the creator's mask.

Creator's mask	New file	New subdirectory	Group can edit file?
007	660	2770	Yes
027	640	2750	No

setgid chooses the group; it does not grant group write. All collaborators need a suitable creation policy.

Assumes file request 666, directory request 777, and no default ACL. To remove directory setgid explicitly: `chmod g-s shared-runs`.

The sticky bit: protecting names in a shared directory

A typical `/tmp` mode is **1777** = `drwxrwxrwt`.

- `777` allows users to traverse the directory and create entries.
- The **sticky bit** restricts deletion and renaming of entries to the file's owner, the directory's owner, or a suitably privileged process.
- Ordinary directory permissions are still required; sticky does not itself grant access.

Deleting a file and modifying its contents are different

Alice owns `/tmp/alice-results.dat`; Bob owns neither that file nor `/tmp`. Bob cannot normally remove or rename Alice's entry.

If the file itself is writable by Bob, he can still modify its contents. The sticky bit does not prevent this.

Compare: setgid controls group inheritance; sticky controls who may remove or rename entries. A directory may use both flags.

Check your reasoning

Assume ordinary Linux permissions, no default ACLs, and typical creation requests: files `666`, directories `777`.

1. A solver has mode `754`. Write its nine permission characters. Which classes may execute it?

2. A process has `umask 023`. Calculate the modes of a new regular file and a new directory. Show the group and others triplets in binary.

3. A shared directory belongs to `physics` and has mode `2770`. Alice creates a file with `umask 027`. What group and mode will it have? Can Bob, also in `physics`, modify its contents?

4. A file has mode `444`, but its containing directory gives you `w+x` and has no sticky bit. Does the file's mode prevent you from deleting its entry?

Section 5

Links and metadata

A filename is not the file itself

A **directory entry** associates a name with an **inode**: the filesystem object that records the file's metadata and identifies its data.

```
ls -li field.dat
stat field.dat
```

- `ls -i`: inode number; `ls -l`: type, permissions, hard-link count, owner, group, size and modification time.
- `stat`: more detail, including several timestamps. Inode numbers are unique only **within one filesystem**.

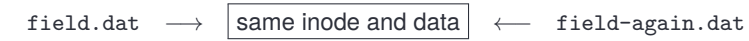
Timestamp	Meaning
`mtime`	Last content modification.
`ctime`	Last inode status change, e.g. writing or changing permissions.
`atime`	Last access; updates may be delayed or disabled.

ctime is not creation time. A separate birth time may be available, depending on the filesystem.

Hard links: two names for the same file

Assume `field.dat` exists and `field-again.dat` does not:

```
ln field.dat field-again.dat
ls -li field.dat field-again.dat
```



- Both names have the same inode number; the hard-link count increases by one. Neither name is a special “original”.
- Writing to the existing file through either name changes the same data. Ownership and permissions are shared too.
- Hard links must stay on the **same filesystem**. Users normally cannot create hard links to directories.

Removing a name does not necessarily remove the data

`rm field.dat` removes one directory entry; `field-again.dat` still works. Space is released after the last hard link is removed and no process keeps the file open.

A hard link is not a backup: it provides no independent copy of the data.

Symbolic links: a stored pathname

A **symbolic link** is a separate file with its own inode. It stores a pathname, which is followed when accessing the target:

```
ln -s field.dat latest.dat
readlink latest.dat
stat latest.dat
stat -L latest.dat
```

- `ls -l` shows `latest.dat -> field.dat`; `readlink` prints the stored pathname.
- GNU `stat` inspects the link itself; `stat -L` follows it and inspects the target.
- Symlinks can point to **directories** and cross **filesystem boundaries**.
- If the target path disappears, the link becomes **dangling**. Removing the link itself leaves its target intact.

A symlink does not bypass permissions

Access still depends on the target and directories along the path. An ordinary Linux symlink's displayed `rwx` bits do not grant access.

Relative symlinks: start from the link's directory

Suppose a project already contains `runs/run-001/field.dat`. From the project directory:

```
ln -s run-001 runs/latest
ls -ld runs/latest
readlink runs/latest
ls runs/latest/
```

`runs/latest/field.dat` ⟶ `runs/run-001/field.dat`

- The stored target `run-001` is interpreted relative to **runs/**, where the link lives, not your current directory.
- An **absolute target** begins with `/`; a relative target can remain valid when the whole project is moved with its internal layout preserved.
- Links let analysis scripts refer to a selected simulation run without duplicating large output files.

Links do not preserve temporary HPC data

A symlink in your home directory pointing into scratch will break when the target data is removed. Copy valuable results to suitable storage before the retention deadline.

Section 6

Environment variables

Shell variables and the process environment

- A **shell variable** stores a value in the current shell. An assignment alone does not export a newly introduced variable.
- An **environment variable** is passed to programs started from the shell. Use `export` to mark a variable for inclusion in that environment.

```
run_label="wave packet"
SIM_DATA="$HOME/unix-course/data"
export SIM_DATA
```

- Assignment: `name=value`, with **no spaces** around `=`. Read a value with `$name` or `${name}`; quote it as `"$name"` in commands.
- Names are case-sensitive. `unset name` removes a variable; `name=""` instead assigns an empty value.

Environment is inherited by child processes

New programs receive a copy of exported values. Changing them does not update already-running jobs or other terminals. A child process cannot change its parent shell's environment.

Inspecting variables: more than echo

Using the variables from the previous slide:

Command	What it displays
`echo "$run_label"`	The value expanded by this shell.
`printf '%s\n' "$run_label"`	The value with explicit output formatting.
`printenv SIM_DATA`	This exported variable's value; supply its name without `$`.
`printenv` or `env`	All environment entries, as `name=value`.
`declare -p run_label`	A Bash variable's value and attributes, exported or not.
`export -p`	Exported variables as shell declarations.
`set`	Shell variables and functions in normal Bash mode; a broader, often much longer listing.

Shell expansion and environment lookup differ

`echo "$run_label"` can display an unexported shell variable. `printenv run_label` finds nothing until it is exported.

Useful environment variables in scientific work

Variable	Meaning or common use
`HOME`, `PWD`	Home directory; current directory tracked by the shell.
`SHELL`	Usually the login shell; may differ from the current shell.
`EDITOR`	Preferred editor for programs that honour this setting.
`LANG`, `LC_ALL`	Locale settings affecting language and formatting. `LC_ALL` overrides the other locale settings.
`OMP_NUM_THREADS`	Configures the default thread count for OpenMP parallel regions.

```
export EDITOR=nano
OMP_NUM_THREADS=4 ./solver
```

`NAME=value command` changes the environment for that invocation only; it leaves the parent shell's value unchanged.

A variable matters only if the program uses it

OpenMP settings do not request CPUs. Match the thread count to your scheduler allocation.

PATH: how the shell finds an external command

PATH is an ordered list of **directories**, separated by colons. An illustrative value:

```
/home/student/.local/bin:/usr/local/bin:/usr/bin:/bin
```

1. You enter `solver`, without a slash. After alias processing, Bash considers functions and builtins.
2. If an external program is needed, Bash searches the `PATH` directories in order, from left to right.
3. The first matching executable is selected. Bash can cache the resolved pathname for later calls.

Directory order can select a different program version

If both `/home/student/.local/bin/solver` and `/usr/bin/solver` exist and are executable, this example selects the first one.

`./solver` contains a slash: it names the executable in the current directory directly and does not search `PATH`.

`PATH` does not specify where simulation data are stored.

Extending PATH and checking command resolution

Add a directory for your own executables, retaining the existing list:

```
mkdir -p "$HOME/.local/bin"
export PATH="$HOME/.local/bin:$PATH"
type -a python3
command -v python3
```

- **Prepend:** your directory is searched first. Appending it with `PATH="$PATH:$HOME/.local/bin"` instead gives existing directories priority.
- `type -a`: shows matching aliases, functions, builtins and executable locations. `command -v`: reports how this shell resolves the name.
- Add a **directory**, not an executable filename. Prefer absolute paths; use `./solver` for a local program instead of adding `.` to `PATH`.

When the wrong Python or solver runs

Check `PATH` order; `hash -r` clears cached locations after an executable is moved.

An empty `PATH` entry, such as in `::`, also means the current directory.

Section 7

Shell configuration

Bash configuration: which file is read?

Startup files contain shell commands: they can configure your prompt, variables, search path and command shortcuts.

Bash session	Normal startup behaviour
Interactive login	Read `/etc/profile`, then the **first** readable file from this list: `~/.bash_profile`, `~/.bash_login`, `~/.profile`.
Interactive non-login	Read `~/.bashrc`, if it exists.
Ordinary script	Does not normally read these interactive startup files; Bash can use `BASH_ENV` instead.

Login and interactive describe different things

Interactive: you enter commands at a prompt. **Login:** a startup mode, commonly used for a login session. A new terminal often starts an interactive non-login shell.

A login profile commonly loads `~/.bashrc` explicitly. Distribution and cluster defaults vary; these rules are for Bash invoked as Bash.

Aliases: short names for frequent commands

In an interactive Bash shell, try:

```
alias ll='ls -lh'
alias la='ls -A'
alias cproj='cd "$HOME/unix-course"'
```

- An alias replaces a command word with text. `ll results.dat` becomes `ls -lh results.dat`.
- `la` includes hidden names, but excludes `.` and `..`. `cproj` changes to the example course directory.
- Use **no spaces around** `=`. Single quotes preserve the replacement text when defining it.
- In `cproj`, `$HOME` is expanded when the alias is used; the inner double quotes protect the resulting path.

An alias is a shell shortcut, not a new program

A definition entered at the prompt belongs to this shell session. It does not create an executable file or modify `PATH`.

Inspecting and controlling aliases

Command	Purpose
`alias`	List the aliases currently defined.
`alias ll`	Show the definition of `ll`.
`type ll`	Explain how Bash interprets the name `ll`.
`type -a ls`	Show matching aliases, functions and command locations.
`unalias ll`	Remove `ll` from the current shell.

To bypass an alias named `ls` for one command:

```
command ls -lh
```

This also bypasses a shell function named `ls`.

Aliases do not have their own parameters

Extra words can follow an alias, as in `ll results.dat`. To use an argument at a chosen position, or implement logic, use a **shell function** or a script.

Keeping aliases for future sessions

Add aliases to the interactive section of `~/.bashrc`; keep existing settings:

```
alias ll='ls -lh'
alias cproj='cd "$HOME/unix-course"'
```

Save the file, then apply it to your current Bash shell:

```
source ~/.bashrc
type ll
```

- `source` (or `.`) executes in the **current shell**; editing alone does not apply changes.
- `bash ~/.bashrc` runs separately and does not configure the parent shell.
- To remove an alias, delete its definition and run `unalias name` in the current shell.

Keep simulation scripts reproducible

Use actual commands in scripts and batch jobs. Bash does not expand aliases in non-interactive mode by default.

Scientific software: modules and project environments

HPC modules: select installed software

On clusters such as Ares, modules configure your environment to use available software, including its `PATH` entries.

```
module avail
module spider python
module load NAME
module list
```

Replace `NAME` with an exact module name and version from the listing. `module unload NAME` removes it from the environment.

Python: install packages for one project

With a suitable Python available, create an isolated environment in your project directory:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install numpy
```

Activation changes `PATH`. Packages go into `.venv`, without `sudo`. Use `deactivate` to leave the environment.

Record the environment used for a calculation

Keep module names, Python and package versions with your results. Load the required modules explicitly in batch scripts. Follow cluster instructions for where to create and install environments.

Conda / Mamba provide another way to manage environments, including Python versions and non-Python libraries.

Basic administration: privileges and system information

`sudo command` runs a permitted command as another user, normally `root`. Access depends on the administrator's policy. `sudo -l` lists your permitted commands.

Command	Useful question
`hostname`	Which machine am I using?
`uptime`	How long has it been running, and what is its load?
`free -h`	How much memory is available? Check the `available` column.
`df -h`	How much space is available on mounted filesystems?
`ps -u "$USER"`	Which processes belong to my user account?
`top`	Which processes are consuming CPU and memory now?

A cluster node is a shared, managed machine

Administrators manage system packages and services. Node-wide resource figures do not describe your job allocation or your personal disk quota.

These inspection commands normally work without `sudo`. When reporting a problem, include the hostname, command and exact error.

Services and logs on a systemd-based machine

On systems using **systemd**, services manage background tasks, such as SSH servers. Replace `NAME.service` with the actual unit name.

Command	Purpose
`systemctl status NAME.service`	Inspect its state and recent messages.
`sudo systemctl start NAME.service`	Start it now.
`sudo systemctl stop NAME.service`	Stop it now.
`sudo systemctl restart NAME.service`	Stop and start it again.
`sudo systemctl enable NAME.service`	Configure it to start at boot.
`journalctl -u NAME.service -b`	Read its logs from the current boot.

Running now and starting at boot are separate settings

`enable` alone does not start a service now. `start` alone does not enable it for future boots. `sudo systemctl enable --now NAME.service` does both.

Some logs require additional privileges. Service names vary by distribution. On HPC clusters, submit simulations through the scheduler; system services belong to the administrators.